

PilotFish Programmable (Trigger) Listener – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **named door into your route**. It does not poll or listen on a network port by itself. Instead, other Java code, the CLI, or another route calls it by name and passes optional data — which starts an integration transaction on demand.

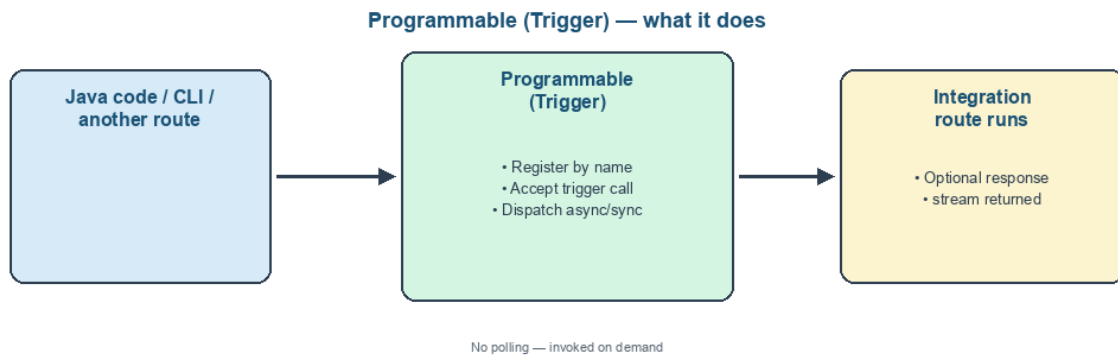


Figure 1: At a glance (plain English)

- **Registers** under the listener's configured name when the route starts.
- **Accepts** programmatic triggers with optional input stream and attributes.
- **Runs** the route asynchronously by default, or waits for a response when asked.
- **Supports** CLI invocation when enabled on the base listener options.

Purpose

Deep dive into the **Programmable (Trigger)** listener in `eip.war.hs.26R1.11`: registration, trigger APIs, synchronous wait, CLI, and configuration.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-core-1.0.3.jar</code>
Core JAR	<code>xcs-core-1.2.6.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.core.Triggerable</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../Tr</code>
Listener type string	<code>Programmable (Trigger)</code>
Description	Programmatically invocable via static methods of the <code>com.pilotfish.eip.modules.core.Triggerable</code> class.
Help ID	<code>console.listener.programmable_java</code>
Category	<code>PILOT_FISH</code>

Related classes:

Class	Role
<code>TriggerableListener</code>	Registration, trigger dispatch
<code>TriggerableListenerUtils</code>	Static <code>trigger</code> / <code>triggerAndWait</code> API
<code>ITriggerableListener</code>	Interface contract
<code>AbstractListener</code>	Base options, synchronous dispatch

1. High-level behavior

1. **startListening**: if `isEnabled()` (default true), registers instance in `NTMTransactionManager` trigger map by listener name; optionally registers with CLI.
2. **stopListening**: unregisters instance; sets `enabled = false`.
3. **External call** via `TriggerableListenerUtils.trigger(name, stream, ...)`:
 - Resolves listener by short name or `InterfaceName::ListenerName`
 - If target has **Initialize on trigger only** and is not listening, calls `startListening()` first
 - Invokes `triggerInternal` on a background thread (async)
4. **triggerInternal**: copies input to default EIP cache (if stream non-null), then `dispatchListeners` on cache input stream.
5. **triggerInternalAndWait**: uses `dispatchForSynchronousResponseTransactionData` with timeout; throws `IllegalStateException("Not currently listening.")` if not enabled.

Route engage -> `startListening` -> `registerInstance(name)`

External `trigger(name, data, attrs)`

```

-> cache copy -> dispatchListeners (async thread)
External triggerAndWait(name, data, timeout)
-> dispatchForSynchronousResponseTransactionData

```

2. Configuration reference

2.1 Module-specific

Label	Tag	Default	Tooltip
Called by	CallingRoutes	""	Routes that call into this listener (informational UI)

`CallingRouteConfigurationItem` — reverse reference for documentation in `eiConsole`; does not affect runtime.

2.2 Inherited (primary operational options)

Label	Tag	Default	Relevance
Initialize on trigger only	IS_TRIGGERABLE_LISTENER	<code>false</code>	When true, listener starts on first trigger
Allow command-line invocation	CLIAAllowed	<code>false</code>	Registers with <code>CLIManager</code> on start
Restart on listening error	RESTART_ON_ERROR_TAG	<code>0</code>	Base listener recovery
FIFO Queue Name / Delay	FIFOQueueName, <code>FIFOQueueDelay</code>	empty / 500	Ordered trigger processing
Throttling *	various	see base	Limits emission rate
Inactivity monitor	various	see base	Alerts if not triggered enough

* Full throttling and inactivity tables match `AbstractListener.getBaseConfiguration()`.

Note: `TriggerableListener.isTriggerable()` returns **false** — the class itself is the trigger endpoint, not a deferred-start consumer of that flag on self. The flag still affects behavior when **other** modules trigger this listener via `TriggerableListenerUtils` (which checks `trig.isTriggerable() && !trig.isListening()`).

3. Static trigger API (`TriggerableListenerUtils`)

Method	Behavior
<code>trigger(name, data, caller)</code>	Async; empty <code>TransactionData</code> attributes
<code>trigger(name, data, original, caller)</code>	Async; clones attributes; callout tracking if caller set
<code>triggerAndWait(name, data, caller, timeout)</code>	Sync stream response
<code>triggerAndWait(name, data, attributes, caller, timeout)</code>	Sync with attributes
<code>triggerAndWait(name, data, timeout)</code>	Sync <code>TransactionData</code> merge back into caller data
<code>listenerExists(name)</code>	Map lookup

Name resolution:

- `MyListener` — first registered match, or match by caller interface context
- `MyInterface::MyListener` — scoped to interface context name

Failure: `EIPEException` with available listener names listed.

4. Dispatch details

4.1 Async `triggerInternal`

1. Requires `enabled == true` else `IllegalStateException(NOT_CURRENTLY_LISTENING)`.
2. Copies stream to `EIPCacheManager.getDefaultCache()` when data non-null.
3. Starts thread "Dispatch Triggerable listeners thread" calling `dispatchListeners(dataFromCache, attributes)`.

4.2 Sync `triggerInternalAndWait`

- Delegates to `AbstractListener.dispatchForSynchronousResponseTransactionData`.
- `triggerAndWait(TransactionData)` variant merges response stream/attributes back into original `TransactionData`, preserving `TransactionInfo`.

5. CLI integration

When **Allow command-line invocation** true and listener starts:

- `CLIManager.getInstance().registerListener(this)` once
- Complements programmatic `TriggerableListenerUtils` access

6. Transaction attributes

No module-specific registered attributes. Callers pass `TransactionAttributes` into trigger methods; attributes are cloned (`cloneAllForResponse`) before dispatch. Standard originating/current listener attributes applied in `initializeData` / `loadTransactionAttributes`.

Testing hook: attribute `com.pilotfish.eip.testingFlag` suppresses actual trigger execution.

7. Worked example (cross-route call)

Interface A has Programmable listener `ProcessOrder`. Interface B Database SQL processor calls:

`TriggerableListenerUtils.trigger("ProcessOrder", orderXmlStream, transactionData, processorMod`

1. Utils resolve `ProcessOrder` in same JVM (prefer same interface context as caller).
2. Input copied to cache; route behind `ProcessOrder` runs asynchronously.
3. Original transaction in Interface B continues without waiting.

For request/response, use `triggerAndWait` with timeout seconds matching route SLA.

8. Known quirks

1. `isTriggerable()` returns false on this class — confusing vs option name; defer-start applies when *invoked through* Utils on listeners with `IS_TRIGGERABLE_LISTENER` true.
 2. **Name changes:** `storeOldName` / `isNameChanged` support re-registration scenarios.
 3. **Empty input allowed:** null stream skips cache copy; dispatch still runs.
 4. **Default cache shared pattern:** each trigger uses default cache — concurrent triggers rely on thread isolation and cache implementation.
 5. **Not currently listening** if route stopped/disabled — triggers fail fast.
 6. **Description references static methods on `TriggerableListener` class** — actual API is `TriggerableListenerUtils`.
-

9. Operational recommendations

Goal	Approach
On-demand only	Use this listener; no polling interval needed
Lazy route startup	Initialize on trigger only = true on <i>target</i> listener config
Sync API from Java	<code>triggerAndWait</code> with explicit timeout
Operations/testing	Enable CLI invocation
Multiple interfaces same name	Use <code>InterfaceName::ListenerName</code> qualifier
Back-pressure	Enable listener throttling + response processor if needed

10. Source index

Topic	Path
TriggerableListener	com/pilotfish/eip/modules/core/TriggerableListener.java
TriggerableListenerUtils	com/pilotfish/eip/TriggerableListenerUtils.java
ITriggerableListener	com/pilotfish/eip/extend/ITriggerableListener.java
AbstractListener	com/pilotfish/eip/extend/AbstractListener.java

Deep-dive documentation from WAR decompile — 26R1.11 — August 2026.